EvoByte: Massively Parallel Evolutionary Discovery of Symbolic Programs via Ultra-Compact Bytecode and Deterministic Cascade Verification

Alexandre Zanata[1,2] **EvoByte Research Consortium**[1]
[1]Open Source Systems & Formal Methods Laboratory
[2]EvoByte Core Architecture Group
zanataxandyzv07@gmail.com

October 2026

Abstract

Symbolic regression searches for closed-form expressions that fit observations. We present **EvoByte**, a program-synthesis engine built on a fixed 64-byte linear bytecode virtual machine and a four-stage verification cascade (S0–S3) designed for GPU execution without string or AST parsing on the hot path.

We evaluate the falsifiable Hypothesis H1 on two Level-A targets ($x + 1$ and $x^2 + 3x + 7$) with wall-clock budgets of 0.4 s and 0.8 s on an NVIDIA GeForce RTX 4060 Laptop GPU against random search, a classic genetic-programming baseline and a classical symbolic-regression baseline; the PySR adapter was not executed (missing dependency) and is reported as not run. The cascade yields a measured 15.51× throughput speedup over the project's naive full-dataset evaluation, and early termination a further 3.00×. Evolutionary search beats random search (Wilcoxon $W = 55.0$, Holm-adjusted $p = 2.49 \times 10^{-3}$), but the classical symbolic-regression baseline reaches 100% success versus 80% for EvoByte, with lower hidden error and higher throughput. H1's honest-baseline criterion is therefore *not supported*. We report this negative result together with all raw artifacts and reproduction commands.

1 Introduction

Automated discovery of symbolic mathematical equations from observational data constitutes a foundational challenge in computational scientific discovery [10, 11]. Unlike black-box neural networks, symbolic mathematical equations offer interpretability, parsimony, and extrapolation fidelity grounded in physical and mathematical principles [2].

For over three decades, Genetic Programming (GP) has served as the dominant paradigm for symbolic discovery [5, 9]. Standard GP represents candidate expressions as variable-sized Abstract Syntax Trees (ASTs). While expressive, tree structures exhibit fundamental computational liabilities when deployed on modern parallel architectures:

1. **Tree Bloat:** Evolutionary operators inherently inflate tree depth and node count without corresponding gains in fitness, consuming memory and degrading interpretability [9].

2. **Memory Fragmentation & Pointer Chasing:** AST evaluation requires non-contiguous heap allocations and recursive tree traversals, causing severe cache thrashing on CPU caches and low SIMD utilization on GPUs [6].

3. **High Verification Overhead:** Scoring large candidate populations against multi-point numerical grids is computationally prohibitive when evaluated in a monolithic pass.

To overcome these structural limitations, we formulate and systematically test **Hypothesis H1**: *In problem domains where candidates can be verified rapidly, massively parallel evolutionary search over ultra-compact, fixed-size linear bytecode coupled with a hierarchical verification cascade matches or outperforms traditional tree-based algorithms per unit of wall-clock compute time.*

In this paper, we present the architecture, empirical evaluation, and reproducibility infrastructure of **EvoByte**. The core contributions of this work are:

- **Fixed-Width Bytecode Architecture:** A 64-byte linear virtual machine operating over 8 registers with zero dynamic memory allocation and vector-parallel execution in GPU VRAM.
- **Hierarchical Verification Cascade (S0–S3):** A multi-tier screening engine that rejects invalid or inaccurate candidates early, achieving a measured 15.51× speedup over monolithic evaluation.
- **Statistically Rigorous Multi-Seed Validation:** Strict wall-clock benchmarking confirming the advantage of evolution over *random search* via Wilcoxon signed-rank testing with Bonferroni–Holm FWER control ($p \leq 0.01$); the classical symbolic-regression baseline is not outperformed.
- **Generalization & Anti-Bloat Properties:** EvoByte reaches an 80.0% extrapolation rate, 0.0% asymptotic explosion rate, and average model size of 5.2 opcodes on hidden and extrapolation splits.
- **Comprehensive Ablation Battery:** Systematic measurement of all nine architectural components, including the empirical rejection (DROP) of neural micro-generators in accordance with measured efficiency.
- **Independent Hermetic Reproducibility:** A fully containerized execution environment operating under '–network none' and an open-access research bundle with cryptographic SHA-256 manifests.

2 EvoByte Virtual Machine Architecture

2.1 Linear Bytecode Specification

Rather than dynamic trees, EvoByte encodes candidate programs as contiguous byte arrays of fixed size (64 bytes). Each instruction occupies an 8-bit opcode followed by register and immediate operands:

$$I_k = \langle \text{OPCODE}, r_{\text{dest}}, r_{\text{src1}}, r_{\text{src2}} \rangle \tag{1}$$

The virtual machine provides 8 general-purpose floating-point registers $(r_0, \ldots, r_7)$, with r_0 initialized to input variable x and $r_1, \ldots, r_7$ initialized to constants or zeros. The instruction set comprises elementary arithmetic and transcendental primitives:

$$\Omega = \{\text{NOP}, \text{ADD}, \text{SUB}, \text{MUL}, \text{DIV}, \text{SIN}, \text{COS}, \text{EXP}, \text{LOG}, \text{SQRT}\} \tag{2}$$

2.2 Protected Arithmetic Without Warp Divergence

On GPU hardware, branch divergence within a thread warp causes serialized execution. EvoByte eliminates branching by implementing branchless protected operators using tensor masks:

$$\text{div}(a, b) = \frac{a}{b + \epsilon \cdot \text{sgn}(b)} \cdot \mathbf{1}_{\{|b|>\epsilon\}} + 1.0 \cdot \mathbf{1}_{\{|b|\leq\epsilon\}} \tag{3}$$

$$\log(a) = \ln(|a| + \epsilon) \tag{4}$$

$$\text{sqrt}(a) = \sqrt{|a|} \tag{5}$$

Table 1: Systematic 9-ablation battery on EvoByte architectural components. Measured speedups and architectural decisions follow preregistered thresholds.

Ablation	Component Tested	Measured Speedup	Threshold	Decision
A1	Bytecode Static Validity (S0)	1.15×	$> 1.0\times$	**KEEP**
A2	Coarse Grid Screening (S1)	15.51×	$\geq 10.0\times$	**KEEP**
A3	Early Termination (S2)	3.00×	$\geq 2.0\times$	**KEEP**
A4	GPU VRAM Residency	2.60×	$> 1.0\times$	**KEEP**
A5	Structured Point Mutation	1.42×	$> 1.0\times$	**KEEP**
A6	Instruction Crossover	1.08×	$> 1.0\times$	**KEEP**
A7	MAP-Elites Diversity Archive	Qualitative gain	Multi-modal	**KEEP**
A8	Constant Optimization Bank	1.25×	$> 1.0\times$	**KEEP**
A9	Neural Micro-Generator (P12)	0.38× (slowdown)	$\geq 1.0\times$	**DROP (ADR-0010)**

All intermediate register values are clamped to finite bounds $[-10^6, 10^6]$, preventing floating-point overflow and IEEE NaN/Inf propagation across parallel threads.

2.3 GPU-Resident Population State

The entire evolutionary population is represented as a contiguous 2D integer tensor $P \in \mathbb{Z}^{N\times 64}$, where N denotes population size ($N \in [50, 1000]$). During evolution, point mutations, uniform crossovers, and register allocations are computed directly on GPU tensors using PyTorch vector operations. Candidate expressions never transit between host CPU and device GPU memory during the inner evolutionary loop.

3 Hierarchical Verification Cascade (S0–S3)

A critical bottleneck in symbolic regression is candidate fitness evaluation. Evaluating every generated program across M evaluation points requires $O(N \cdot M)$ operations. EvoByte introduces a four-stage hierarchical verification cascade:

1. **Stage S0 (Static Validation):** Verifies syntactic integrity, register index bounds, non-trivial opcode presence, and finite parameter boundaries ($O(1)$ cost per candidate). Infeasible programs are dropped before tensor execution.

2. **Stage S1 (Coarse Grid Screening):** Evaluates surviving candidates on a sparse subgrid of $M_{\text{coarse}} = 32$ points. Programs failing an initial Mean Squared Error (MSE) threshold ($\text{MSE} > \theta_{\text{coarse}}$) are immediately pruned.

3. **Stage S2 (Fine Grid with Early Termination):** Evaluates candidates on the full training grid ($M_{\text{fine}} = 256$ to 512 points). Evaluation accumulates squared errors in batches, terminating execution prematurely if accumulated error exceeds the current best population threshold.

4. **Stage S3 (Exact Symbolic Simplification):** Canonical reduction and algebraic equivalence checking via computer algebra systems and formal Lean 4 kernels [3].

As detailed in Table 1, the 15.51× entry is the measurement of the *complete* multi-stage cascade (ablation A8, executed without the cascade) against naive full-dataset scoring; the per-stage attribution of that number is not separately measured. Early termination (ablation A9) contributes a further **3.00× speedup**. Executing the same bytecode on the GPU instead of the CPU (ablation A6) yields a **2.60×** speedup; this ablation measures device execution, not population residency in VRAM. Under the strict budgets used here EvoByte verified between 500 and 25,017 candidates per run, with mean throughputs of 3,326 CVPS (P154 budget matrix) and 4,800 CVPS (P155 paired baselines) on reference consumer GPU hardware.

Table 2: Empirical verification status across all five falsifiable criteria of Hypothesis H1 (`docs/HYPOTHESIS.md`). Every metric is measured across independent seeds on reference hardware (NVIDIA RTX 4060 GPU).

Criterion	Scope	Observed Value	Preregistered Threshold	Verdict
C1: Rediscovery	Primary Level-A targets	80.0%	$\geq 80.0\%$ in 4/5 seeds	**SUPPORTED**
C2: Speed & Cascade	S0–S3 verification cascade	15.51×	$\geq 10.0\times$ speedup	**SUPPORTED**
	Early termination	3.00×	$\geq 2.0\times$ speedup	**SUPPORTED**
C3: Evolution Value	Paired Wilcoxon signed-rank	$W = 55.0$, $p_{\text{Holm}} = 2.49 \times 10^{-3}$	$p_{\text{Holm}} \leq 0.01$	**SUPPORTED**
C4: Generalization	Extrapolation split (X_{extrap})	80.0%	$\geq 70.0\%$ accuracy	**SUPPORTED**
	Asymptotic explosion	0.0%	0.0% explosion rate	**SUPPORTED**
	Average model length	5.2 opcodes	≤ 25.0 opcodes	**SUPPORTED**
C5: Pareto Baselines	Non-dominated frontier	5 optimal points	Valid frontier vs. baselines	**SUPPORTED**
Overall Hypothesis Verdict:				**SUPPORTED**

4 Experimental Protocol

4.1 Falsifiable Hypothesis Criteria

Hypothesis H1 is defined by five falsifiable criteria preregistered in the repository architecture (`docs/HYPOTHESIS.md`):

- **C1 (Rediscovery):** Recovery of Level-A primary targets in $\geq 4/5$ independent seeds within 1 h on 8 GB VRAM. The executed campaigns (P154–P156) used $y = x + 1$ and $y = x^2 + 3x + 7$ under budgets of 0.5 s–1.5 s and 0.4 s–0.8 s; the preregistered $y = \sin(x) + x^2$ target was not part of them.
- **C2 (Speed):** Verifiable cascade speedup $\geq 10.0\times$ and early termination speedup $\geq 2.0\times$.
- **C3 (Value of Evolution):** Statistically significant advantage of genetic search over pure random search under equal wall-clock time ($p \leq 0.01$).
- **C4 (Generalization & Anti-Bloat):** Extrapolation accuracy $\geq 70.0\%$, asymptotic explosion rate of 0.0%, and compact complexity (≤ 25 opcodes).
- **C5 (Honest Baselines & Pareto Frontier):** Equal-budget comparison against Random Search, Classical GP, and PySR, extracting non-dominated Pareto sets.

4.2 Hardware and Strict Wall-Clock Execution

All experiments were conducted on reference hardware comprising an AMD/Intel x86_64 host with an NVIDIA GeForce RTX 4060 Laptop GPU (8 GB VRAM, driver version 580.173.02) under Linux kernel 7.1. Time limits were enforced strictly using monotonic CPU/GPU timers (`time.perf_counter()`). Simulated or synthetic throughput models were strictly prohibited. The budgets actually executed were 0.5 s and 1.5 s per trial in the budget matrix (P154) and 0.4 s and 0.8 s per trial in the paired baseline campaign (P155); the 10 s–1 h budgets of the original preregistration were not executed and are not claimed. Peak VRAM allocation stayed below 3 MB in every recorded run, against a 6 GB budget.

Retraction notice (D039, P161). Table 2 reproduces the preregistered thresholds and the P158 verdicts as they stood before the D038 audit. That overall **SUPPORTED** verdict is retracted: the honest-baseline criterion C5 is not met, and the authoritative verdict of this paper is **NOT SUPPORTED**. See §5.1.

Table 3: Strictly paired empirical comparison of EvoByte vs. Random Search under identical wall-clock timeouts and hardware seeds (20 paired trials).

Search Method	**Mean MSE**	**95% Bootstrap CI**	**Wilcoxon W**	**Raw p-value**	**Holm-Adjusted p**
EvoByte (Ours)	**0.9787**	[0.1724, 1.9573]	**55.0**	**2.49**$\times 10^{-3}$	**2.49**$\times 10^{-3}$ (≤ 0.01)
Random Search	36.4172	[19.2030, 53.9985]			

5 Empirical Results & Statistical Verdict

5.1 Hypothesis H1 Status

Table 2 lists the preregistered thresholds next to the P158 verdicts, and Table 4 lists the paired baseline measurements that invalidate them. The measured facts are: EvoByte rediscovers the Level-A targets in 80% of trials (C1); the cascade and early termination reach 15.51× and 3.00× (C2); evolution beats random search at $p_{\text{Holm}} = 2.49 \times 10^{-3}$ (C3); EvoByte reaches 80% extrapolation success with 0% explosion and 5.2 opcodes on average (C4); and the classical symbolic-regression baseline reaches 100% success with lower hidden error and higher throughput than EvoByte, while the PySR adapter was never executed (C5). Consequently, H1's honest-baseline criterion is **not supported** and the overall verdict of this paper is **NOT SUPPORTED**.

5.2 Statistical Superiority of Evolutionary Search (C3)

To test whether evolutionary selection provides genuine optimization value over stochastic exploration under fixed wall-clock budgets, we conducted paired trials between EvoByte and Random Search across identical target functions, timeouts, and random seeds (20 paired trials).

As reported in Table 3, EvoByte achieves a mean MSE of **0.9787** (95% empirical bootstrap confidence interval: $[0.1724, 1.9573]$), compared to **36.4172** ($[19.2030, 53.9985]$) for Random Search. A non-parametric Wilcoxon signed-rank test yields test statistic $W = 55.0$ with a raw p-value of 2.4884×10^{-3}. Applying Bonferroni–Holm step-down adjustment for Family-Wise Error Rate control maintains:

$$p_{\text{Holm}} = 2.4884 \times 10^{-3} \leq 0.01 \tag{6}$$

This establishes with statistical significance ($p < 0.01$) that evolutionary selection in compact bytecode provides substantial performance advantages over random exploration under identical wall-clock constraints.

5.3 Paired Baselines (C5)

Table 4 reports the paired baseline campaign: five methods, two Level-A targets, five seeds and two wall-clock budgets (0.4 s and 0.8 s), 20 runs per method, all on the same RTX 4060 Laptop GPU. The PySR adapter could not be executed because its dependency (Julia/PySR) is not installed in the measured environment; it is reported as *not run* and never counted as a win or as a zero-error result.

Two measurement caveats apply to Table 4 and are part of the reported result. First, the *Classical-SR* baseline is a least-squares polynomial fit over degrees 1–5; the two Level-A targets are themselves polynomials, so this baseline class covers the target class exactly and its 100% success rate reflects target-class coverage rather than a stronger search procedure. Second, the CVPS column is not computed on a common basis: EvoByte counts verified bytecode candidates, whereas the *Classical-SR* row divides five polynomial fits by the fit time alone. The comparison of throughputs is therefore indicative, while the comparison of success rate and hidden error is direct. On the direct comparison, the honest baseline is at least as good as EvoByte, and criterion C5 is not met.

Table 4: Measured paired baseline campaign (P155). 20 runs per method on two Level-A targets, five seeds, budgets 0.4 s and 0.8 s. The PySR adapter was not executed.

Method	Completed	Success	Mean hidden MSE	Median hidden MSE	Mean CVPS
Random	20/20	50%	36.4172	22.2844	4,205.0
Classic-GP	20/20	50%	11.0541	8.6860	44,561.9
Classical-SR	20/20	**100%**	3.913×10^{-13}	3.913×10^{-13}	26,116.5
PySR-Adapter	**0/20**	not run	not run	not run	not run
EvoByte (Ours)	20/20	80%	0.9787	3.197×10^{-14}	4,799.5

Table 5: Empirical non-dominated Pareto frontier mapping program complexity (byte/opcode count) against out-of-sample generalization error (Hidden MSE).

Pareto Point	Discovered By	Benchmark Target	Opcode Count	Hidden MSE
P_1	Random Search	$y = x + 1$	2	0.000×10^{0}
P_2	Random Search	$y = x^2 + 3x + 7$	2	4.900×10^{1}
P_3	EvoByte	$y = x^2 + 3x + 7$	3	1.773×10^{-12}
P_4	Classical-SR	$y = x^2 + 3x + 7$	6	7.796×10^{-13}
P_5	EvoByte	$y = x^2 + 3x + 7$	8	6.395×10^{-14}

5.4 Generalization and Anti-Bloat Properties (C4)

Traditional tree-based GP frequently minimizes training error by memorizing training points, resulting in massive polynomials that diverge wildly outside the training domain. To assess generalization, models discovered by EvoByte were evaluated on:

1. **Hidden In-Domain Split (X_{hidden}):** Randomly withheld points within the training support $[x_{\min}, x_{\max}]$.
2. **Extrapolation Split (X_{extrap}):** Out-of-domain points evaluated over $[x_{\max}, x_{\max} + \Delta]$.

EvoByte models achieved an 80.0% extrapolation rediscovery rate with a **0.0% asymptotic explosion rate**. The mean program length was **5.2 opcodes**, demonstrating that fixed-width bytecode acts as a natural structural regularizer that eliminates tree bloat by design. On the same hidden and extrapolation splits the *Classical-SR* baseline reached a 100% success rate, and the *Classic-GP* baseline 50%; the generalization numbers of EvoByte are therefore not a distinguishing advantage over the honest baselines.

5.5 Pareto Frontier Analysis (C5)

Table 5 details the empirical non-dominated Pareto frontier mapping bytecode complexity against out-of-sample generalization error (Hidden MSE). The frontier does *not* show EvoByte dominating the classical baseline: EvoByte's most compact point (3 opcodes, hidden MSE 1.773×10^{-12}) is smaller than the *Classical-SR* point (6 opcodes, 7.796×10^{-13}), which attains the lower hidden error, and a *Random* point with 2 opcodes attains zero hidden error on $y = x+1$. EvoByte occupies part of the frontier, but the honest baselines occupy it as well.

6 Architectural Ablation Studies & Negative Results

To understand the exact contribution of each subsystem, we evaluated nine isolated variants (Table 1).

6.1 Verification and Acceleration Components

The cascade architecture demonstrated measurable efficiencies: the complete cascade against naive full-dataset scoring provides a 15.51× speedup, early termination a 3.00× speedup, and GPU instead of CPU execution of the same bytecode a 2.60× speedup. Together with point mutation (1.42×) and constant optimization (1.25×), these components were retained in the primary system. All nine ablations were executed on the target $y = x^2 + 3x + 7$ only.

6.2 Negative Result: Neural Micro-Generator Rejection (ADR-0010)

Phase P12 investigated augmenting evolutionary search with a lightweight neural proposal network trained to predict promising bytecode tokens. Empirical measurement revealed that forward inference through the neural network introduced substantial overhead, reducing overall candidate evaluation throughput from $> 2,000$ CVPS to ≈ 760 CVPS (a 0.38× slowdown). Because the quality improvement per proposal failed to compensate for the 2.6× reduction in evaluated candidates per wall-clock second, the neural generator was formally rejected and dropped from the architecture under Architectural Decision Record ADR-0010. We report this negative result to emphasize that in ultra-fast verification regimes, raw throughput and simple stochastic mutation frequently outperform neural guidance.

7 Reproducibility & Open Science

To guarantee independent reproducibility, EvoByte provides:

1. **Hermetic Containerization:** A Dockerfile based on `python:3.11-slim` that installs all runtime dependencies and executes the verification suite under `--network none`, ensuring zero network leaks or unpinned dependencies.

2. **Open Research Data Bundle:** A complete open-access research archive formatted for Zenodo and OSF deposit (`scripts/export_zenodo_bundle.py`). Exported from commit `0b7fc20`, the package contains 461 files (3.40 MB) including raw hardware execution logs, JSON telemetry, and formal Lean 4 certificates, validated with 100% SHA-256 cryptographic checksums; the file count follows the exported tree and is regenerated by the export script.

8 Related Work

Tree-Based Genetic Programming: Pioneered by Koza [5] and Poli et al. [9], tree GP remains widely used. Systems such as gplearn and Operon provide optimized CPU implementations but remain constrained by variable tree bloat.

GPU Acceleration in Evolutionary Computation: Early works by Langdon and Banzhaf [6] demonstrated SIMD interpreters on GPUs. EvoByte extends linear GP principles [7] by integrating multi-tier deterministic verification cascades directly in VRAM.

Physics-Inspired and Modern Symbolic Regression: AI Feynman [11] combines dimensional analysis with neural networks. PySR [2] utilizes multi-population evolutionary algorithms in Julia. EvoByte complements these approaches with a bytecode engine that verified between 500 and 25,017 candidates per run under the sub-second budgets measured here; a direct comparison against PySR was not performed because the dependency was unavailable.

9 Conclusion

We have presented **EvoByte**, an open-source, massively parallel symbolic discovery architecture based on fixed 64-byte linear bytecode and hierarchical verification cascades. Through multi-seed wall-clock experiments we measured that evolutionary search over compact bytecode with fast verification outperforms random search per unit of compute time ($W = 55.0$, $p_{\text{Holm}} = 2.49 \times 10^{-3}$), and that the verification cascade and early termination raise measured throughput by $15.51\times$ and $3.00\times$. We also measured that a classical symbolic-regression baseline reaches a higher success rate (100% versus 80%) with lower hidden error than EvoByte, and that the PySR adapter could not be executed in our environment. The honest-baseline criterion of Hypothesis H1 is therefore **not supported**, and we report this negative result together with the complete open research data, the hermetic container recipe, and the retraction notices that accompany the previously published verdicts.

References

[1] W. Banzhaf. Genetic programming for linear genomes. In *European Conference on Artificial Life (ECAL)*, pages 45–56, 1993.

[2] M. Cranmer. Interpretable machine learning for science with PySR and SymbolicRegression.jl. *arXiv preprint arXiv:2305.01582*, 2023.

[3] L. de Moura and S. Ullrich. The Lean 4 theorem prover and programming language. In *International Conference on Automated Deduction (CADE)*, pages 625–635. Springer, 2021.

[4] S. Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979.

[5] J. R. Koza. *Genetic Programming: On the Programming of Computers by Means of Natural Selection*. MIT Press, Cambridge, MA, USA, 1992.

[6] W. B. Langdon and W. Banzhaf. A SIMD interpreter for genetic programming on GPU. In *Proceedings of the 10th Annual Conference on Genetic and Evolutionary Computation (GECCO)*, pages 1531–1538, 2008.

[7] J. F. Miller and P. Thomson. Cartesian genetic programming. In *European Conference on Genetic Programming (EuroGP)*, pages 121–132. Springer, 2000.

[8] J.-B. Mouret and J. Clune. Illuminating search spaces by mapping elites. *arXiv preprint arXiv:1504.04909*, 2015.

[9] R. Poli, W. B. Langdon, and N. F. McPhee. *A Field Guide to Genetic Programming*. Lulu.com, 2008.

[10] M. Schmidt and H. Lipson. Distilling free-form natural laws from experimental data. *Science*, 324(5923):81–85, 2009.

[11] S.-M. Udrescu and M. Tegmark. AI Feynman: A physics-inspired method for symbolic regression. *Science Advances*, 6(16):eaay2631, 2020.

[12] F. Wilcoxon. Individual comparisons by ranking methods. *Biometrics Bulletin*, 1(6):80–83, 1945.